ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

ORGANIZATION OF ISLAMIC COOPERATION (OIC)

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

Project Report

Course: Microcontroller Based System Design Lab

Course Code: EEE 4706

Project Title: Real Time Clock

Group: G1 (C1)

Submitted By:

Muhtasim Sharar - 200021317

Maimuna Biswas Noshin - 200021347

Safrina Kabir- 200021341
Taseen Ishraque Choudhury- 200021361
Tashfin Azwad Siddique- 200021335

Real Time Clock (RTC) using 8051 Microcontroller

Introduction

This project focuses on designing and implementing a **Real Time Clock (RTC)** using the **8051 microcontroller**. The RTC displays the current time in a digital format on an LCD and allows the user to set the time manually via a keypad. Additionally, the system supports **AM/PM** display and **resetting** using external inputs.

Objectives

- To display real-time hours, minutes, and seconds on an LCD.
- To allow manual time setting via a 4x4 matrix keypad.
- To support 12-hour format with AM/PM indication.
- To demonstrate use of external interrupts for reset and mode selection.
- To utilize buzzer notifications on transition from AM to PM.

Required Components

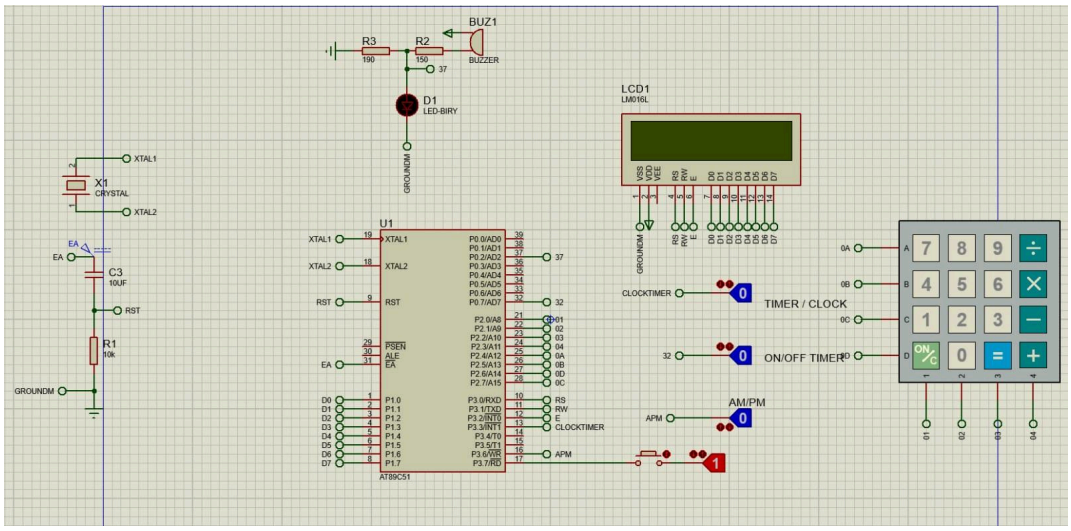
Hardware Requirements

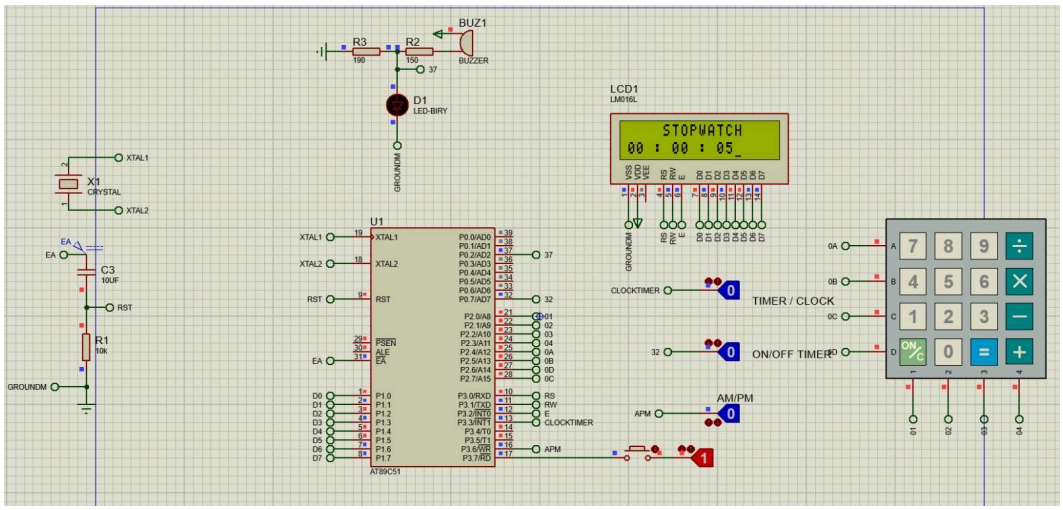
- **Microcontroller:** AT89C51 (8051 Family)
- **LCD Display:** 16x2 alphanumeric LCD
- **Matrix Keypad:** 4x4 Keypad
- **Buzzer/LED:** Connected to port pin for notifications
- **Push buttons:** For reset and mode selection
- **Miscellaneous:** Resistors, capacitors, breadboard, connecting wires, power supply

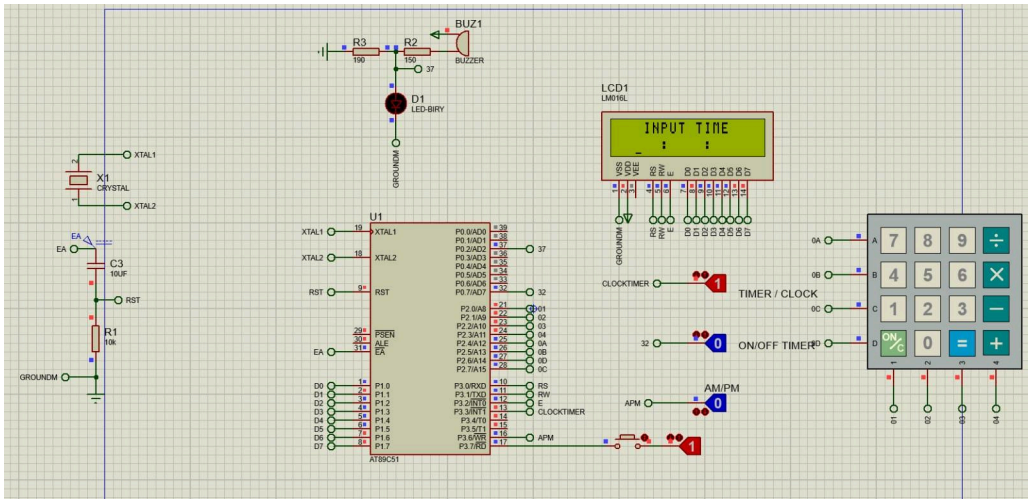
Software Requirements

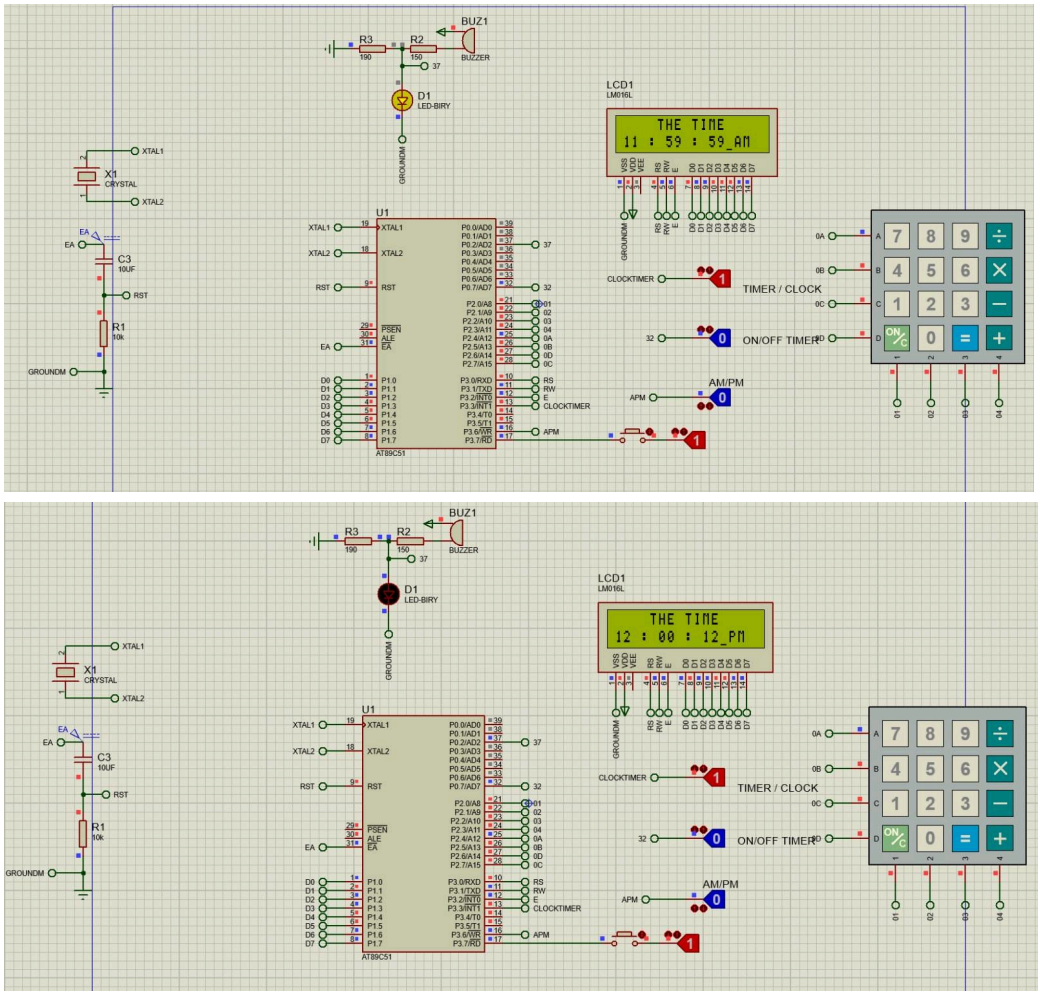
- M-IDE studio for MCS-51
- Proteus

Circuit Diagram









Explanation:
This circuit is a digital clock and timer based on the AT89C51 microcontroller. It uses a 16x2 LCD to display the clock, timer, and stopwatch information, while a keypad allows users to input time settings and switch between modes like clock, timer, and stopwatch. A crystal oscillator provides a stable clock for accurate timing, and reset circuitry ensures proper MCU startup. External switches handle mode selection and AM/PM setting for 12/24-hour formats. When a timer or alarm triggers, a buzzer sounds and an LED lights up as an alert. Overall, it's a simple and practical 8051-based project for timekeeping applications.

Working Principle

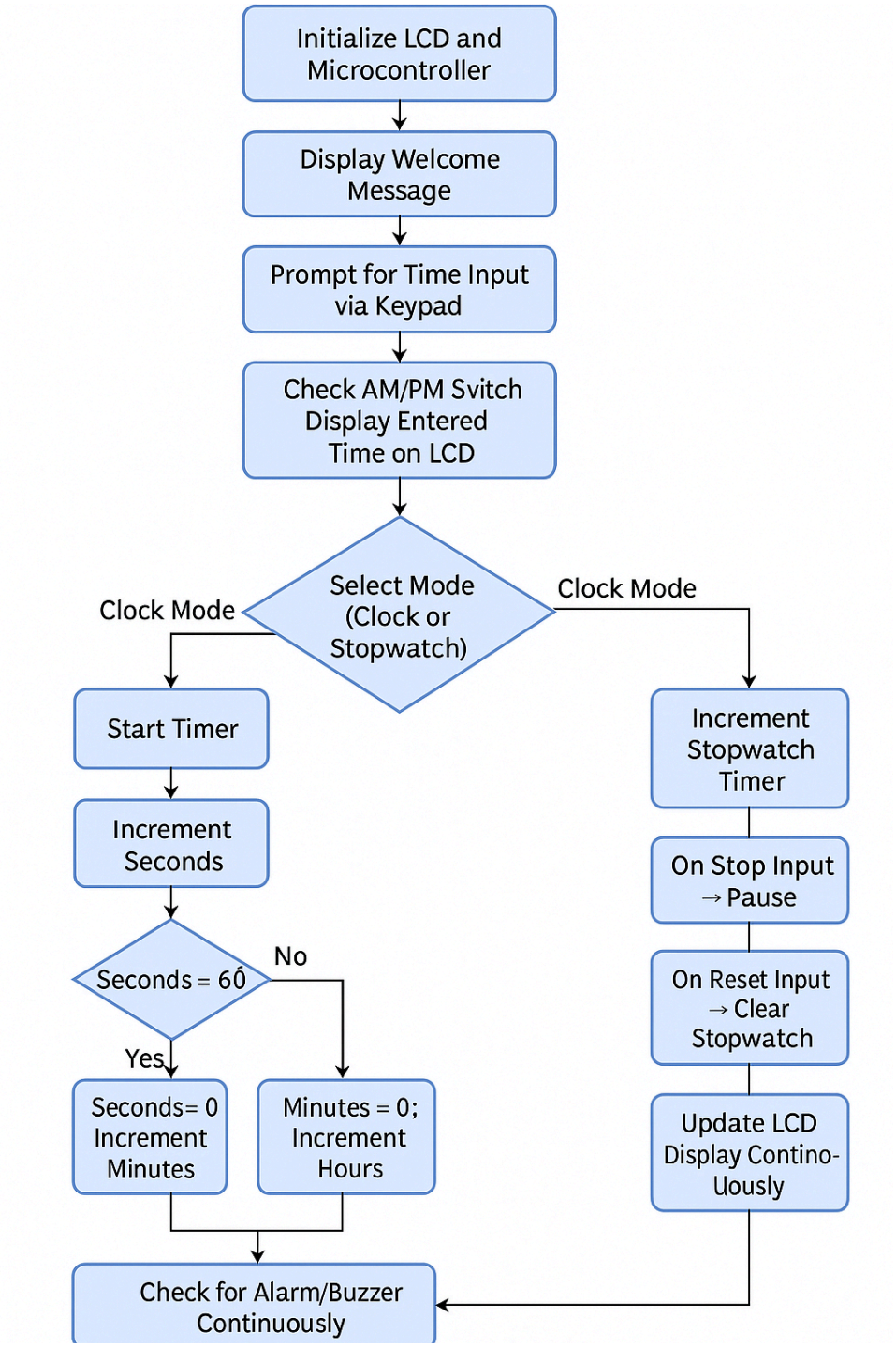
This project uses an AT89C51 microcontroller to build a digital clock and stopwatch system. At startup, the system first initializes the LCD by following a specific sequence (stored in the `MYLCD` table) to get it ready for displaying messages. It checks the AM/PM setting using a switch connected to pin P3.6 — if the input is low, it starts with AM; if high, it starts with PM.

The user can manually set the initial time by pressing keys on a 4x4 keypad connected to Port 1. Each keypress is detected using a row-column scanning method, decoded, and displayed neatly on the LCD. The digits for hours, minutes, and seconds are saved individually in internal

memory locations (40H–45H). Once the time is set, the clock runs automatically, continuously updating seconds, minutes, and hours.

The clock handles overflow smartly: when seconds reach 60, they reset to 00 and increment the minute; 60 minutes reset to 00 and increment the hour. For 12-hour format management, after 12:59:59, the system toggles the AM/PM status and triggers a buzzer and LED as a signal. The LCD keeps showing a clear display with the header “TIME NOW,” along with the current time (HH:MM:SS) and AM/PM indication. Cursor positions are carefully controlled for a neat look.

There’s also a stopwatch mode: by switching modes, the system lets you start, stop, and reset the stopwatch independently without affecting the clock. Throughout everything, the buzzer alerts when certain milestones are reached, and the microcontroller ensures real-time operation by continuously looping through the timer logic, display updates, and keypad reading with software debouncing for reliability.



Code

```
        ORG 00H                          ; Starting the code execution from the origin address 00H
          LJMP MAIN                      ; Jump to the MAIN section of the program

MAIN:   MOV DPTR,#LCDINITIALIZATION  ; Setting up Data Pointer (DPTR) for LCD initialization sequence
        ;Assigning specific pins for the peripherals
```

```
        PORT EQU P1             ; Port 1 is used for general output (LCD)
        BUZZ EQU P0.2           ; Buzzer connected to pin P0.2
        STOPPIN EQU P0.7        ; Stop pin (used for stopping the operation and reset) at P0.7
        RS EQU P3.0             ; RS pin for LCD control at P3.0
        RW EQU P3.1             ; RW pin for LCD read/write control at P3.1
        E  EQU P3.2             ; Enable pin for LCD at P3.2
        CLKTM EQU P3.3                  ; Clock pin at P3.3
    APM EQU P3.6                ; AM/PM switch at P3.6
        RSTPN EQU P3.7                  ; Reset pin at P3.7
        ; Check AM/PM switch and set 'B' register accordingly
    JNB APM, J1                 ; If APM is not set, jump to J1 (AM function)
    MOV B,#11111111B    ; else PM (B = 11111111B) B register is used to store AM flag and PM flag
    SJMP J2                     ; Jump to J2 to continue with the remaining portion

J1:   MOV B, #00000000B         ; AM (B = 00000000B) flag initialization


;Initialize the I/O pins
J2:     CLR  CLKTM      ; Clear clock pin (disable clock)
        CLR BUZZ        ; Clear buzzer pin (turn off buzzer)
    CLR RSTPN; Clear reset pin
;DISPLAY INITIALIZATION
J3:   CLR A             ; Clear the accumulator
        MOVC A,@A+DPTR      ; Move content from DPTR into A (fetch initialization byte)
        LCALL COMMAND       ; Call LCD command function subroutine
        LCALL DELAY     ; Call delay for proper timing
        JZ TIMER            ; If zero, jump to TIMER to start the stopwatch
        INC DPTR            ; Increment the data pointer to fetch next initialization byte for lcd
        SJMP J3                 ; Jump back to J3 until end of the sequence 0 is reached

TIMER:  JB CLKTM,S1     ; If clock pin is set, jump to S1 (start stopwatch)
    MOV DPTR ,#MY_ST ; Set DPTR to "MY_ST" (stopwatch title)
TS1:   CLR A            ; This function allows the LCD to show the stopwatch title
        MOVC  A,@A+DPTR ; Fetch content for display from DPTR
        LCALL DISPLAY  ; Display fetched data on LCD
        LCALL DELAY     ; Call delay for refresh
        JZ TS2   ; If zero, jump to TS2 (stopwatch value initiation)
        INC DPTR            ; Increment DPTR to fetch the next byte
        SJMP TS1            ; Jump back to TS1 to keep fetching display content until zero is reached
TS2:    ; Clear memory locations for time digits initialization to be 00:00:00
        MOV 40H, #0H    ; Second Right Digit (SR) Ones digit
        MOV 41H, #0H    ; Second Left Digit (SL) Tens digit
        MOV 42H, #0H    ; Minute Right Digit (MR) Ones digit
        MOV 43H, #0H    ; Minute Left Digit (ML) Tens digit
        MOV 44H, #0H    ; Hour Right Digit (HR) Ones digit
        MOV 45H, #0H    ; Hour Left Digit (HL) Tens digit
        ; Initialize register pointers for time storage
        MOV R0,45H      ;SR is stored at R0
        MOV R1,44H      ;SL is stored at R1
        MOV R2,43H      ;MR is stored at R2
        MOV R3,42H      ;ML is stored at R3
        MOV R4,41H      ;HR is stored at R4
        MOV R5,40H      ;HL is stored at R5

        LCALL DELAY     ; Add delays for LCD refresh
        LCALL DELAY
TS3:    JB STOPPIN, TS3             ; Wait for stop signal to start the operation
        LJMP J7             ; Jump to J7 for further execution
S1:
    MOV DPTR,#INPUTMESSAGE ; DPTR stores the input message string for the clock initialization
D1:
        CLR A
```

```
        MOVC  A,@A+DPTR                 ; Fetch next byte of the input message
        LCALL DISPLAY           ; Display it on the LCD
        LCALL DELAY             ; Delay for LCD
        JZ S2                   ; If the message ends, jump to S2
        INC DPTR                ; Increment DPTR to the next character
        SJMP D1                         ; Continue fetching and displaying message

;Placing the colons in the proper position

S2:   MOV DPTR,#COLON  ;DPTR stores the colon sign :
        MOV A,#0C5H     ;Position C5 on the lcd stores the colon sign
        LCALL COMMAND    ;The position is sent to the lcd using command

        CLR A
        MOVC  A,@A+DPTR         ;the colon is loaded to A register
        LCALL DISPLAY   ;the colon sign is displayed

        MOV A,#0CAH     ;Position CA on the lcd stores the colon sign
        LCALL COMMAND
        ;similarly we place one more colons
        CLR A
        MOVC  A,@A+DPTR
        LCALL DISPLAY

    CLR A
    MOV A,#0C2H    ;Position C2 on the lcd to start placing our digits
        ACALL COMMAND
        ACALL DELAY

        LCALL KEYPAD    ;Keypad subroutine is called to input the value of the hour left digit
        LCALL DISPLAY   ;displays the digit on the lcd
        LCALL DELAY
        ANL A,#0FH      ;Using AND operation on the lower and upper nibble so that the upper nibble is always
zero
        MOV 40H,A       ;saving the digit in the memory location
        ; simillarly we store all the digits for the hours and minutes and seconds in location C3, C7, C8, CC and CD
        ; all the digits are stored sequencially in 41H 42H 43H 44H AND 45H
        CLR A
    MOV A,#0C3H
        ACALL COMMAND
        ACALL DELAY

        LCALL KEYPAD
        LCALL DISPLAY
        LCALL DELAY
        ANL A,#0FH
        MOV 41H,A

    CLR A
    MOV A,#0C7H
        ACALL COMMAND
        ACALL DELAY

        LCALL KEYPAD
        LCALL DISPLAY
        LCALL DELAY
        ANL A,#0FH
        MOV 42H,A

        CLR A
    MOV A,#0C8H
        ACALL COMMAND
```

```
        ACALL DELAY

        LCALL KEYPAD
        LCALL DISPLAY
        LCALL DELAY
        ANL A,#0FH
        MOV 43H,A

        CLR A
    MOV A,#0CCH
        ACALL COMMAND
        ACALL DELAY

        LCALL KEYPAD
        LCALL DISPLAY
        LCALL DELAY
        ANL A,#0FH
        MOV 44H,A

        CLR A
    MOV A,#0CDH
        ACALL COMMAND
        ACALL DELAY

        LCALL KEYPAD
        LCALL DISPLAY
        LCALL DELAY
        ANL A,#0FH
        MOV 45H,A

        SJMP START              ; Jump to start the clock
TX1:    JNB CLKTM , TX1                 ; loop for pausing the stopwatch
        LJMP MAIN               ; if the flag is one then the program re initializes
;CLOCK DIGIT INITIALIZATION
;These are the memory addresses for each digit on the clock (seconds, minutes, hours).
START:  MOV R0,45H    ;SR  Second Right Digit (R0 points to address 45H)
        MOV R1,44H    ; SL  Second Left Digit (R1 points to address 44H)
        MOV R2,43H    ; MR  Minute Right Digit (R2 points to address 43H)
        MOV R3,42H    ; ML  Minute Left Digit (R3 points to address 42H)
        MOV R4,41H    ; HR  Hour Right Digit (R4 points to address 41H)
        MOV R5,40H    ; HL  Hour Left Digit (R5 points to address 40H)

        LCALL DELAY
        LCALL DELAY
        MOV A, #01      ; Clear the LCD display (A = 01 indicates clearing
        ACALL COMMAND   ;call command subroutine
        ACALL DELAY     ;give LCD DELAY

        MOV A, #80H     ; Move the value to set cursor at line 1, position 4 (LCD command)
        ACALL COMMAND
        ACALL DELAY

S_DATA2:
        MOV DPTR,#TIMEMESSAGE ; Load the address of TIMEMESSAGE into the Data Pointer (DPTR)


;function to print the time title
D2:
        CLR A
        MOVC  A,@A+DPTR
        LCALL DISPLAY
        LCALL DELAY
```

```
        JZ S_DATA               ; If zero flag is set (end of string), jump to S_DATA to print next line.
        INC DPTR
        SJMP D2

S_DATA:
    JB CLKTM , J8               ; If CLKTM (clock timer flag) is set, jump to J8 that is start clock else start
stopwatch
        JB STOPPIN , TX1        ; If STOPPIN is set, jump to TX1 (restart initialization) where it will wait for the
next command
        SJMP J7                         ; jump to J7 to increment the time
J8:   MOV DPTR,#MSGM            ;This portion sets the AM or PM shown in the display we select the M letter and
display it
        MOV A,#0CEH             ; M letter is displayed at position CE
        LCALL COMMAND


        CLR A
        MOVC  A,@A+DPTR
        LCALL DISPLAY

    CLR A                       ; Clear a to check for the AM PM flag
    MOV A,B                     ; Take the APM value
    JZ J5                       ; If zero AM
    SJMP J6                     ; Else PM
J5:   MOV DPTR,#MSGA            ;stores the A letter
        MOV A,#0CDH             ;displays the letter at CD
        LCALL COMMAND


        CLR A
        MOVC  A,@A+DPTR
        LCALL DISPLAY

        SJMP J7                         ; After the A is shown it jumps to the incrementing portion of the code
J6:   MOV DPTR,#MSGP            ;stores the P letter
        MOV A,#0CDH             ; Displays P instead of A at CD position
        LCALL COMMAND

        CLR A
        MOVC  A,@A+DPTR
        LCALL DISPLAY

        SJMP J7                 ;Start the Increment


J7:   MOV DPTR,#COLON           ;DPTR stores the colon sign :
        MOV A,#0C3H             ;Change the position of the Hour Minute and Second to make space for AM/PM
        LCALL COMMAND

        CLR A
        MOVC  A,@A+DPTR
        LCALL DISPLAY

        MOV DPTR,#COLON
        MOV A,#0C8H
    LCALL COMMAND

    CLR A
    MOVC  A,@A+DPTR
    LCALL DISPLAY
    ; These lines below involve displaying the current values of hours, minutes, and seconds.
     MOV DPTR,#NUMBER           ; load the ascii values of the numbers
```

```
        MOV A,#0C0H                     ; the left hour digit
        LCALL COMMAND
        LCALL DELAY

        MOV A,R5            ; the value of A stores the stored value for the left hour digit
        MOVC  A,@A+DPTR             ; due to the preloaded value at A we shift the DPTR which has the initial location
of Numbers 0 we correspondingly shift slightly to get the number that is stored at R5
        LCALL DISPLAY
        LCALL DELAY
        ; we do the same for the next 5 digits
        MOV A,#0C1H
        LCALL COMMAND
        LCALL DELAY

        MOV A,R4
        MOVC  A,@A+DPTR
        LCALL DISPLAY
        LCALL DELAY


        MOV A,#0C5H
        LCALL COMMAND
        LCALL DELAY

        MOV A,R3
        MOVC  A,@A+DPTR
        LCALL DISPLAY
        LCALL DELAY

        MOV A,#0C6H
        LCALL COMMAND
        LCALL DELAY

        MOV A,R2
        MOVC  A,@A+DPTR
        LCALL DISPLAY
        LCALL DELAY


        MOV A,#0CAH
        LCALL COMMAND
        LCALL DELAY

        MOV A,R1
        MOVC  A,@A+DPTR
        LCALL DISPLAY
        LCALL DELAY

        MOV A,#0CBH
        LCALL COMMAND
        LCALL DELAY


        MOV A,R0
        MOVC  A,@A+DPTR
        LCALL DISPLAY
        LCALL DELAY3

W1:     JNB RSTPN,W2            ;If the reset Pin is high we will clear the display and go back to the MAIN label to
initialize again else we jump to incrementation of the digits at W2
        MOV A,#01H
        LCALL COMMAND
```

```
        LCALL DELAY
        LJMP MAIN           ;Jump to INITIALIZE

W2 :     INC R0             ; Increments the second value
        CJNE R0,#10,L2      ; comparing that it has not become 10 if not 10 jump to L2
        SJMP L9             ; else jump to L9 where we will move 0 to R0 and increment the next digit R1 for our left
second digit
L9:      MOV R0,#0

        INC R1
        CJNE R1,#6,L2               ; check due to incrementing whether the Left second digit has become 6 or not. If
not go to L2 to go back to the updating time part in the LCD
        SJMP L10            ; Jump to L10 to increment the right minute digit
L10:     MOV R1,#0                  ; moves 0 to the left second digit and increments the minute

        INC R2              ; increment the minute right digit
        CJNE R2,#10,L2      ; checks whether it is more than 10 or not. If not then go to S_Data
        SJMP L7             ; else increment the next digit
L7:      MOV R2,#0                  ; Resets the value of the right digit to 0 again

        INC R3              ; increment the minute left digit
        CJNE R3,#6,L2               ;check due to incrementing whether the Left minute digit has become 6 or not. If
not go to L2 to go back to the updating time part in the LCD
        SJMP L8             ; Otherwise, jump to L8
L8:      MOV R3,#0                  ; Reset minute left digit
        INC R4              ; Increment hour right digit
        LJMP G2             ; Jump to G2 to do hour logic check

L2:      LJMP S_DATA                ; Jump back to S_DATA to continue updating time


L5:     MOV R4,#0
       MOV R5,#0
       SJMP L6
L6:    LJMP S_DATA

G2:  CJNE R5,#1,MODE_2                  ; If R5 is not 1, jump to MODE_2
        SJMP MODE_3                 ; Otherwise, jump to MODE_3

MODE_2:
        CJNE R4,#10,Q2      ; If R4 is not 10, jump to Q2 that is go back to updating time
        SJMP Q3             ; Else jump to Q3 to update the R5
Q2:      LJMP S_DATA

Q3:      MOV R4,#0                  ; Reset hour right digit
        INC R5              ; Increment hour left digit
        CJNE R5,#1,Q6               ; If R5 is not 1, jump to Q6 that is go back to counting
        LJMP S_DATA
MODE_3:  CJNE R4,#2,J9              ; If R4 is not 2, jump to J9
       CLR A                        ;clear register A
       MOV A,B              ; move the B flag AM PM flag
       CPL A                        ;complement it
       MOV B,A              ; move the changed AM PM flag back to B
            SETB BUZZ               ; start the 12 hour buzzer
       LCALL DELAY3                 ; call a delay for the buzzer
       CLR BUZZ             ;clear the buzzer
J9:     CJNE R4,#3,Q4               ; If R4 is not 3, jump to Q4 to go back to counting
       SJMP Q5              ; go for the hour reset to 01:00:00
Q4:     LJMP S_DATA
Q5:     MOV R4,#1                   ; Set hour right digit to 1
       MOV R5,#0            ; Set hour left digit to 0
      SJMP Q6
```

```
Q6:     LJMP S_DATA


KEYPAD:
 MOV A,#0FH
 MOV P2,A   ;MAKE P2 = INPUT
 K1:
 MOV P2,#00001111B
 MOV A,P2  ;READ ALL COLUMNS, ENSURE ALL KEYS OPEN
 ANL A,#00001111B   ;MASK UNUSED BITS
 CJNE A,#00001111B,K1  ;CHECK TILL ALL KEYS RELEASED
 K2:
 ACALL DELAY  ;CALL DELAY
 MOV A,P2 ;SEE IF ANY KEY IS PRESSED
 ANL A,#00001111B  ;MASK UNUSED BITS
 CJNE A,#00001111B,OVER  ;KEY PRESSED, AWAIT CLOSURE
 SJMP K2  ;CHECK IS KEY PRESSED
 OVER:   ACALL DELAY   ;CALL DELAY
 MOV A,P2   ;CHECK KEY CLOSURE
 ANL A,#00001111B  ;MASK UNUSED BITS
 CJNE A,#00001111B,OVER1  ;KEY PRESSED, FIND RO
 SJMP K2  ;IF NONE, KEEP POLLING
 OVER1: MOV P2,#11101111B ;GROUND ROW 0
 MOV A,P2  ;READ ALL COLUMNS
 ANL A,#00001111B   ;MASK UNUSED BITS
 CJNE A,#00001111B,ROW_0  ;KEY ROW 0, FIND THE COLUMN
 MOV P2,#11011111B  ;GROUND ROW 1
 MOV A,P2  ;READ ALL COLUMNS
 ANL A,#00001111B   ;MASK UNUSED BITS
 CJNE A,#00001111B,ROW_1   ;KEY ROW 1, FIND THE COLUMN
 MOV P2,#01111111B  ;GROUND ROW 2
 MOV A,P2  ;READ ALL COLUMNS
 ANL A,#00001111B  ;MASK UNUSED BITS
 CJNE A,#00001111B,ROW_2    ;KEY ROW 2, FIND THE COLUMN
 MOV P2,#10111111B   ;GROUND ROW 3
 MOV A,P2   ;READ ALL COLUMNS
 ANL A,#00001111B  ;MASK UNUSED BITS
 CJNE A,#00001111B,ROW_3   ;KEY ROW 3, FIND THE COLUMN
 LJMP K2   ;IF NONE, FALSE INPUT, REPEAT
 ROW_0: MOV DPTR,#KCODE0  ;SET DPTR=START OF ROW 0
 SJMP FIND  ;FIND COLUMN.KEY BELONGS TO
 ROW_1: MOV DPTR,#KCODE1  ;SET DPTR=START OF ROW 1
 SJMP FIND   ;FIND COLUMN.KEY BELONGS TO
 ROW_2: MOV DPTR,#KCODE2 ;SET DPTR=START OF ROW 2
 SJMP FIND   ;FIND COLUMN.KEY BELONGS TO
 ROW_3: MOV DPTR,#KCODE3  ;SET DPTR=START OF ROW 3
 FIND: RRC A   ;SEE IF ANY CY BIT IS LOW
 JNC MATCH  ;IF ZERO, GET THE ASCII CODE
INC DPTR   ;POINT TO THE NEXT COLUMN ADDRESS
 SJMP FIND ;KEEP SEARCHING
 MATCH: CLR A  ;SET A=0 (MATCH FOUND)
 MOVC A,@A+DPTR  ;GET ASCII CODE FROM TABLE
 RET   ;RETURN TO CALLER
 COMMAND:
 MOV PORT,A   ;COPY REG A TO PORT
 CLR RS    ;RS=0 FOR COMMAND
 CLR RW  ;R/W=0 FOR WRITE
 SETB E   ;E=1 FOR HIGH PULSE
 ACALL DELAY   ;GIVE LCD SOME TIME
 CLR E  ;E=0 FOR H-TO-L PULSE
 RET   ;RETURN TO CALLER
 DISPLAY:
```

```
 MOV PORT,A  ;COPY REG A TO PORT
 SETB RS   ;RS=1 FOR DATA
 CLR RW    ;R/W=0 FOR WRITE
 SETB E    ;E=1 FOR HIGH PULSE
 ACALL DELAY   ;GIVE LCD SOME TIME
 CLR E    ;E=0 FOR H-TO-L PULSE
 RET   ;RETURN TO CALLER


;Delay for display and data processing
DELAY:  SETB PSW.2       ;SELECTING REGISTER BANK 1 AS TO NOT CONFLICT THE REGISTER BANK 0
VALUES R0 TO R5 WHICH HAVE OUR TIME DETAILS
       MOV R6, #10
H1:    MOV R7,#5            ;NORMAL DELAY FOR THE LCD
H2:    DJNZ R7,H2
       DJNZ R6,H1
       CLR PSW.2
       RET

;Delay for Buzzer
DELAY1: SETB PSW.4
       MOV R5,#235
H5:    MOV R6, #7           ;BUZZER DELAY
H3:    MOV R7,#230
H4:    DJNZ R7,H4
       DJNZ R6,H3
       DJNZ R5,H5
       CLR PSW.4
            RET

;Real time delay
DELAY3: SETB PSW.3
       MOV R5,#46
H6:    MOV R6,#67
H7:    MOV R7,#148
H8:    DJNZ R7,H8     ;1 SEC DELAY
       DJNZ R6,H7
       DJNZ R5,H6
       CLR PSW.3
       RET
;ASCII LOOK-UP TABLE FOR EACH ROW
KCODE0: DB '7','8','9','/' ;ROWnumber 0
KCODE1: DB '4','5','6','*' ;ROWnumber 1
KCODE2: DB '1','2','3','-' ;ROWnumber 2
KCODE3: DB 99H,'0','=','+' ;ROWnumber 3

ORG 350H
INPUTMESSAGE: DB "   INPUT TIME",0
TIMEMESSAGE: DB "    THE TIME",0
LCDINITIALIZATION : DB 38H,0EH,01,06,80H,0
COLON: DB ":"
NUMBER: DB "0","1","2","3","4","5","6","7","8","9"
MSGA: DB "A"
MSGP: DB "P"
MSGM: DB "M"
MY_ST: DB "    STOPWATCH",0
END
```

Hardware Implementation

Problems Faced

While working on our 8051-based digital clock and stopwatch project, we encountered several challenges that tested our problem-solving skills and pushed us to dig deeper into both the hardware and software aspects of the system.
The key issues we faced:

1. Difficulty in Storing Clock Data

One of the first hurdles we hit was figuring out how to reliably store the clock's time data (hours, minutes, and seconds). The 8051 microcontroller has limited registers, and we were using R0 to R5 to hold the digits for hours (HL, HR), minutes (ML, MR), and seconds (SL, SR). However, we ran into conflicts because other parts of the program—like the delay routines (DELAY, DELAY1, DELAY3)—also used registers R5, R6, and R7 for loop counters. This overlap caused the time data to get overwritten, leading to incorrect time displays.

Solution: We realized we needed to isolate the registers used for time storage from those used in other routines. To fix this, we switched register banks in the delay routines by setting the Program Status Word (PSW) bits (SETB PSW.2 for DELAY, SETB PSW.3 for DELAY3, SETB PSW.4 for DELAY1). This allowed us to use a different bank (Bank 1, 2, or 3) for the delay counters, preserving the time data in Bank 0's R0 to R5.

2. AM/PM Flag Management

Another issue we faced was managing the AM/PM indicator for the 12-hour clock format. Initially, we used the B register to store the AM/PM state (00000000B for AM, 11111111B for PM), but we didn't have a proper mechanism to toggle or update this flag consistently. This caused the AM/PM display to either not change when it should (e.g., at 12:00) or to display incorrectly during initialization.

Solution: We introduced a dedicated check for the AM/PM flag using the APM pin (P3.6) at the start of the program (JNB APM, J1). We also added logic in the MODE_3 section to toggle the flag (CPL A, MOV B, A) when the hours transitioned from 12 to 1, ensuring the AM/PM state updated correctly. However, we found that the flag needed to be set before the clock was initialized for it to work properly, which led to the next issue.

3. AM/PM Flag Initialization Dependency

We noticed that the AM/PM flag had to be set before the clock initialization for the system to work as expected. If we tried to change the AM/PM state after the clock started running, the display wouldn't update correctly, and the toggle logic wouldn't kick in until the next 12-hour cycle. This was frustrating because it meant we couldn't dynamically switch between AM and PM during runtime without resetting the entire system.

Solution: To address this, we ensured the AM/PM flag was set at the very beginning of the program (MAIN section) based on the APM pin input. We also added a reset mechanism (JNB RSTPN, W2) that jumps back to MAIN when the reset pin (P3.7) is triggered, allowing us to

reinitialize the clock with the correct AM/PM state. While this worked, it highlighted a limitation in our design—we couldn't change the AM/PM state on the fly without a full reset.

4. Separate Inputs for Stopwatch

The stopwatch feature was a core part of our project, but we struggled with how to manage its inputs separately from the clock. Initially, both the clock and stopwatch shared the same input pins and logic, which caused conflicts. For example, pressing a key to set the clock time would also interfere with the stopwatch's operation, leading to unpredictable behavior like the stopwatch starting or stopping unexpectedly.

Solution: We introduced a dedicated pin, CLKTM (P3.3), to switch between clock and stopwatch modes. When CLKTM is set, the program jumps to the clock initialization (S1), and when it's cleared, it runs the stopwatch (TIMER). We also used STOPPIN (P0.7) to control the stopwatch's start/stop functionality (JB STOPPIN, TS3). This separation helped, but we still faced issues with the stopwatch's operation, as described below.

5. Stopwatch Input Conflicts with Clock

Even after separating the inputs, we found that the stopwatch's operation conflicted with the clock. The stopwatch could only be started once per run, and if we tried to switch back to the clock mode after starting the stopwatch, the system would either freeze or reset unexpectedly. This was because the stopwatch loop (TS3) and the clock loop (S_DATA) shared the same time digit registers (40H to 45H), and the stopwatch's increment logic interfered with the clock's timekeeping.

Solution: We made the stopwatch and clock modes mutually exclusive by ensuring that the program either runs in stopwatch mode (TIMER) or clock mode (S1), but not both simultaneously. We also added a check (JB CLKTM, J8) to decide which mode to run and used TX1 to loop back to MAIN for reinitialization if needed. However, this meant the stopwatch could only be started once per run, and we couldn't seamlessly switch between modes without resetting the system. But we can go to clock mode from stopwatch mode.

6. Hour Update Issues

Updating the hour values proved to be a significant challenge, especially when transitioning between 12-hour cycles. For example, when the hours reached 12, the system was supposed to reset to 1 (e.g., 12:59:59 to 01:00:00) and toggle the AM/PM flag. However, we kept running into issues where the hours would either increment indefinitely (e.g., to 13, 14, etc.) or reset incorrectly (e.g., to 00 instead of 01).

Solution: We implemented a detailed hour update logic in the G2, MODE_2, and MODE_3 sections. Specifically:

- In MODE_2, we checked if the hour right digit (R4) reached 10 (CJNE R4, #10, Q2), and if so, we reset it to 0 and incremented the hour left digit (R5).
- In MODE_3, we handled the 12-hour boundary by checking if R4 reached 2 (indicating 12:xx:xx) and toggling the AM/PM flag (CPL A, MOV B, A). If R4 reached 3, we reset the hours to 01:00:00 (MOV R4, #1, MOV R5, #0). This fixed most of the hour update issues, but it required careful testing to ensure the transitions were smooth.

7. Hardware Challenges

On the hardware side, we faced a lot of problems. The 8051 development board and its peripherals were finicky to work with. The LCD sometimes wouldn't display properly, likely due to timing issues or loose connections. The keypad occasionally registered false inputs, especially when we pressed keys too quickly, and the buzzer would sometimes stay on longer than intended, which was annoying during debugging.

Solution: We tackled these hardware issues through trial and error:

- For the LCD, we fine-tuned the delay routines (DELAY, DELAY3) to ensure proper timing for commands and data writes.
- For the keypad, we added debouncing logic in the KEYPAD subroutine (OVER, OVER1) to filter out false inputs.
- For the buzzer, we adjusted the DELAY1 routine to control its duration more precisely (MOV R5, #235, MOV R6, #7, MOV R7, #230). While these fixes got the hardware working, they highlighted the importance of robust hardware design and the need for better documentation from the hardware side of things.

Conclusion

This project successfully demonstrates the implementation of a real-time clock using an 8051 microcontroller with LCD and keypad interfacing. It showcases important embedded system concepts such as I/O interfacing, timer management, and interrupt handling.

The system is fully functional, user-interactive, and robust for real-time operation.